期中報告：fork Client-Server 系統資訊服務

Nov 10, 2026 · @ansen

一、系統設計說明

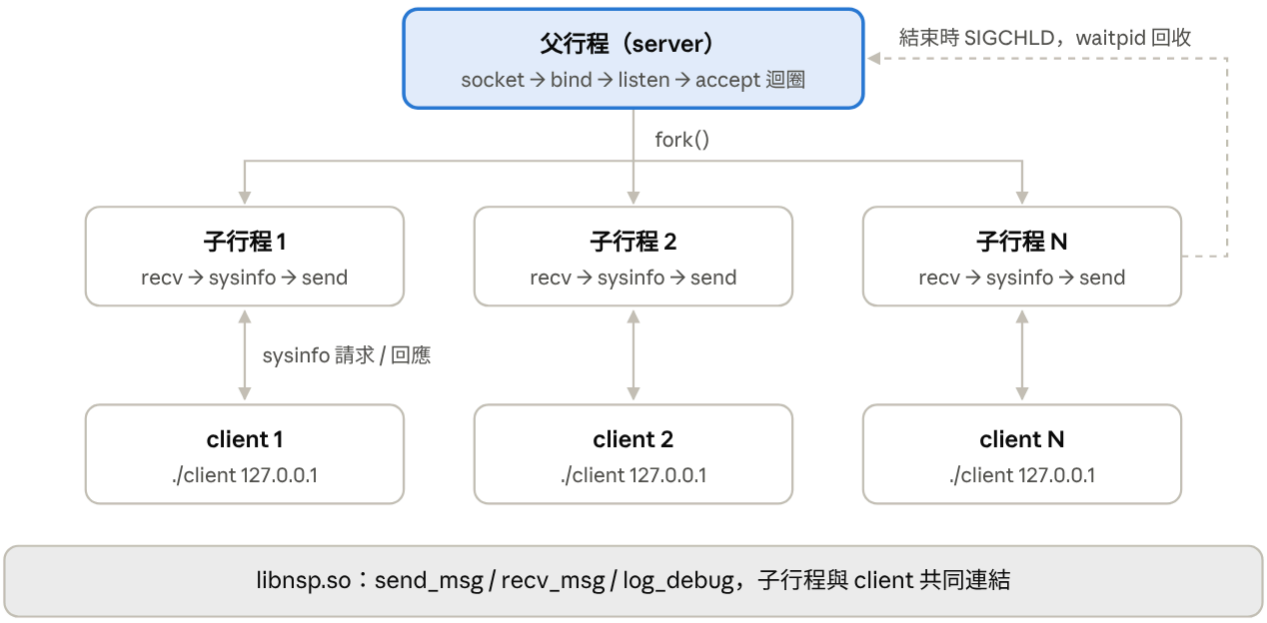
本系統由一支 server、一支 client 和一個共用動態函式庫 `libnsp.so` 組成。server 每接受一個 TCP 連線就 `fork()` 一個子行程服務該 client，父行程只負責接受新連線，因此多個 client 可以同時被服務。

元件

元件	檔案	職責
共用函式庫	`nsp.h`、`nsp.c` → `libnsp.so`	送出與接收訊息（`send_msg`、`recv_msg`）、除錯 log（`log_debug`）
Server	`server.c`	監聽 port 9000；每個連線 fork 一個子行程；查詢系統資訊並回傳；3 項防護機制
Client	`client.c`	連到 server、送出 `sysinfo` 請求、印出回應
併發測試	`test10.sh`	同時啟動 10 個 client 並列出行程樹與連線
建置	`Makefile`	編譯函式庫與兩支程式

運作流程

1. Server 建立 socket，`bind` 到 port 9000，`listen` 等待連線。
2. Client `connect` 到 server。
3. Server 的 `accept` 回傳新連線後立刻 `fork()`：
    a. 子行程關閉監聽 socket，接收請求，查詢系統資訊，回傳，然後 `exit`。
    b. 父行程關閉這條連線的 socket，回到 `accept` 等下一個 client。
4. Client 收到回應後印出並結束。
5. 子行程結束時核心送出 `SIGCHLD`，父行程在處理函式中用 `waitpid` 回收它。



fork 架構 · 1 個父行程、N 個子行程

父行程永遠回到 `accept`，所以同時有幾個 client，就有幾個子行程在工作；虛線是子行程結束後的回收路徑。

訊息協定

每則訊息都是「長度 + 資料」，接收方先讀長度，就知道後面還要讀多少位元組，不會因為 TCP 把資料拆段或黏在一起而讀錯。

欄位	大小	內容
長度	4 位元組	資料的位元組數，網路位元組順序（big-endian）
資料	可變，最多 4095 位元組	請求為字串 `sysinfo`；回應為系統資訊文字

一次完整的交換是：client 送出一則 `sysinfo` 請求，server 回傳一則系統資訊，然後 server 關閉連線。

回傳的系統資訊

主機名稱、核心版本、開機秒數、1/5/15 分鐘平均負載、可用與總記憶體、行程總數，以及服務這個 client 的子行程 PID。資料來自 `uname()` 與 `sysinfo()` 系統呼叫。

二、題目要求對照

題目的 9 項規定全部實作，每一項都有可重現的證明方式；對應的截圖見第六節。

規定	做法	證明方式
用 `fork()` 處理每個連線，不用 thread	`accept` 後 `fork()`，子行程服務 client	程式碼；`ps --forest` 行程樹
同時處理 ≥ 10 個 client	client 可指定「連上後等幾秒再送請求」，讓連線同時存在	`test10.sh`：1 個父行程 + 10 個子行程、`ss` 列出 10 條 ESTABLISHED 連線
封包擷取與分析	`tcpdump -i lo` 錄製，`tshark` 解碼	第五節逐封包分析
編譯期 debug log 開關	`make DEBUG=1` 加上 `-DDEBUG`，`log_debug` 的內容才會被編進去	一般編譯時即使設定環境變數也不印
執行期 debug log 開關	環境變數 `DEBUG_LOG=1` 才印出	DEBUG 版本未設定時不印、設定後印出 `[DEBUG pid ...]`
共用函式封裝成 `.so`	`send_msg`、`recv_msg`、`log_debug` 編成 `libnsp.so`	`ldd server client` 兩者都連到 `libnsp.so`
client 向 server 要系統資訊	`sysinfo` 請求 → `uname()` + `sysinfo()`	client 執行結果
強健性機制，≥ 3 組有／無對照	3 項防護；`./server norobust` 關閉防護	第四節
Makefile 或 CMake	`Makefile`	`make` 一次編出全部

三、關鍵程式碼說明

核心只有四段：server 的 fork 迴圈、子行程回收、共用函式庫的訊息收送，以及兩層 debug log。

1. Server 的 fork 迴圈（`server.c` 的 `main`）

```
while (1) {
    peer_len = sizeof(peer);
    client_fd = accept(listen_fd, (struct sockaddr *)&peer, &peer_len);
    if (client_fd < 0) {
        perror("accept");
        continue;
    }
    pid = fork();
    if (pid == 0) {
        close(listen_fd);
        printf("[pid %d] serving %s:%d\n", getpid(),
inet_ntoa(peer.sin_addr), ntohs(peer.sin_port));
        handle_client(client_fd, robust);
        close(client_fd);
        printf("[pid %d] done\n", getpid());
        exit(0);
    }
    close(client_fd);
}
```

`fork()` 之後父、子行程各有一份 socket。子行程關掉用不到的監聽 socket，只服務這個 client；父行程關掉這條連線的 socket，立刻回到 `accept`。所以父行程不會被任何一個 client 卡住。

2. 子行程回收（`reap_children`）

```
static void reap_children(int sig)
{
    while (waitpid(-1, NULL, WNOHANG) > 0) {
    }
}
```

一個 `SIGCHLD` 可能代表多個子行程同時結束（訊號不排隊），所以用迴圈把所有已結束的都收掉；`WNOHANG` 讓沒有可收的就立刻返回，不會卡住。註冊時加 `SA_RESTART`，讓被訊號打斷的 `accept` 自動重來。

3. 訊息收送（`nsp.c`）

```
int recv_msg(int fd, char *buf, int size)
{
    uint32_t net_len;
    int len;

    if (read_all(fd, (char *)&net_len, 4) < 0) {
        return -1;
    }
    len = ntohl(net_len);
    if (len < 0 || len >= size) {
        return -1;
    }
    if (read_all(fd, buf, len) < 0) {
        return -1;
    }
    buf[len] = '\0';
    return len;
}
```

TCP 是位元組串流，一次 `recv` 不保證讀到完整訊息。`read_all` 用迴圈讀到指定的位元組數為止；`recv_msg` 先讀 4 位元組長度，檢查不超過緩衝區，再讀資料。`write_all` 同理，並用 `MSG_NOSIGNAL`，對方已斷線時回傳錯誤，而不是收到 `SIGPIPE` 被殺掉。

4. 兩層 debug log（`log_debug`）

```
void log_debug(const char *fmt, ...)
{
#ifdef DEBUG
    va_list args;
    char *env = getenv("DEBUG_LOG");

    if (env == NULL || strcmp(env, "1") != 0) {
        return;
    }
    fprintf(stderr, "[DEBUG pid %d] ", getpid());
    va_start(args, fmt);
    vfprintf(stderr, fmt, args);
    va_end(args);
    fprintf(stderr, "\n");
#endif
}
```

第一層是 `#ifdef DEBUG`：沒有用 `make DEBUG=1` 編譯時，函式內容整段不存在。第二層是 `getenv("DEBUG_LOG")`：即使編進去了，執行時也要設 `DEBUG_LOG=1` 才會印。

四、強健性機制

Server 有 3 項防護；執行 `./server norobust` 會關閉全部防護，同一支程式就能對照有、無的差異。以下輸出都來自實際執行。

#	異常狀況	沒有防護	有防護	做法
1	子行程結束後沒人回收	3 個 client 後留下 3 個 `Z <defunct>`	一個都沒有	`SIGCHLD` handler + `waitpid(WNOHANG)`
2	client 連上後不送請求	子行程一直卡在 `recv`	5 秒後斷線並印出 `no valid request (timeout ...)`	`SO_RCVTIMEO` = 5 秒
3	server 重新啟動	`bind: Address already in use`，約 60 秒內無法重啟	立刻重啟成功	`SO_REUSEADDR`

防護 1：回收子行程

子行程結束後，核心保留它的結束狀態，等父行程用 `wait` 系列取走；在取走之前它就是殭屍行程。Server 是長間執行的程式，每服務一個 client 就多一個殭屍，最後會用光行程表。

```
# 沒有防護：./server norobust，執行 3 次 client 後
    PID    PPID STAT CMD
   2059    2056 Z    [server] <defunct>
   2061    2056 Z    [server] <defunct>
   2063    2056 Z    [server] <defunct>

# 有防護：./server，同樣操作
    PID    PPID STAT CMD
```

防護 2：等待請求逾時

子行程用 `recv` 等待請求，預設會無限期等下去。一個連上但不說話的 client（程式卡住、網路中斷或惡意佔用）會讓子行程永遠不會結束。實測用 `./client 127.0.0.1 8`，讓 client 連上後等 8 秒才送請求。

```
# 沒有防護：第 6 秒時子行程仍在等
    PID STAT CMD
   2197 S    ./server norobust

# 有防護：第 5 秒 server 主動斷線
[pid 1932] serving 127.0.0.1:38712
[pid 1932] no valid request (timeout or disconnected)
[pid 1932] done
# client 端：
send failed (server closed the connection)
```

防護 3：埠重用

Server 回應後先關閉連線，按照 TCP 規定，先關閉的一方要在 `TIME_WAIT` 狀態停留約 60 秒。這段時間內，沒有 `SO_REUSEADDR` 的程式無法重新綁定 port 9000。

```
# 沒有防護：server 服務過 client 後重啟
TIME-WAIT 0      0          127.0.0.1:9000      127.0.0.1:58980
bind: Address already in use

# 有防護：同樣操作
server pid 1948 listening on port 9000 (robust=1)
```

五、封包擷取與分析

一次請求共 14 個封包：3 個三向交握、8 個資料與確認、3 個關閉連線，資料部分正好對應自訂協定的「4 位元組長度 + 資料」。在 VM 的 loopback 介面用 tcpdump 錄製：

```
sudo tcpdump -i lo -nn -w midterm.pcap tcp port 9000
./client 127.0.0.1
tshark -r midterm.pcap
```

客戶端埠為 47002，伺服器埠為 9000。

#	方向	旗標	資料長度	意義
1	client → server	SYN	0	三向交握：client 要求連線（`connect`）
2	server → client	SYN, ACK	0	三向交握：server 同意
3	client → server	ACK	0	三向交握完成，`accept` 回傳
4	client → server	PSH, ACK	4	請求的長度欄位：`00 00 00 07` = 7
5	server → client	ACK	0	確認收到

#	方向	旗標	資料長度	意義
6	client → server	PSH, ACK	7	請求資料：`sysinfo`
7	server → client	ACK	0	確認收到
8	server → client	PSH, ACK	4	回應的長度欄位：180
9	client → server	ACK	0	確認收到
10	server → client	PSH, ACK	180	回應資料：系統資訊文字
11	client → server	ACK	0	確認收到
12	server → client	FIN, ACK	0	server 子行程 `close`，先關閉
13	client → server	FIN, ACK	0	client `close`
14	server → client	ACK	0	連線結束；server 端進入 `TIME_WAIT`

觀察：

- 長度與資料各自成為一個封包（4→6、8→10），因為 `send_msg` 分兩次 `send`。接收端用 `read_all` 讀滿指定位元組，所以拆成幾個封包都不影響正確性。
- 是 server 先送 FIN（第 12 個），所以 `TIME_WAIT` 留在 server 端，這就是防護 3 要解決的問題。
- 資料以明文傳輸，用 `tshark -r midterm.pcap -x` 可以直接看到 `sysinfo` 與回應內容。

六、執行截圖

以下 12 張圖取自同一次完整 demo 的終端機錄影（`video/demo.mp4`），每張圖上方是 server 視窗，下方是 client 與指令視窗。

圖 1：編譯與共用函式庫。 `make` 一次編出 `libnsp.so`、`server`、`client`；`ldd` 顯示兩支程式都連結到 `libnsp.so`。

```
$
```

```
$ # 1. 編譯：Makefile 產生 libnsp.so、server、client
$ make clean; make
rm -f libnsp.so server client
gcc -Wall -fPIC -shared -o libnsp.so nsp.c
gcc -Wall -o server server.c -L. -lnsp -Wl,-rpath,'$ORIGIN'
gcc -Wall -o client client.c -L. -lnsp -Wl,-rpath,'$ORIGIN'
$ ldd server client | grep nsp
        libnsp.so => /Users/ansenchen/learndata/NetworkSystemSecurity/midterm/./libnsp.so (0x0000f6eeb0f80000)
        libnsp.so => /Users/ansenchen/learndata/NetworkSystemSecurity/midterm/./libnsp.so (0x0000fd1c5bb20000)
$
```

圖 2：client 取得系統資訊。 上方 server 印出服務這個 client 的子行程 PID，與 client 收到的 `served by: pid` 一致。

```
$ ./server
server pid 34516 listening on port 9000 (robust=1)
[pid 34558] serving 127.0.0.1:35142
[pid 34558] done
```

```
$ # 2. client 向 server 查詢系統資訊（server 用 fork 的子行程服務）
$ ./client 127.0.0.1
hostname : lima-nsp
kernel   : Linux 6.8.0-142-generic
uptime   : 871 seconds
load avg : 0.12 0.13 0.11
memory   : 2176 MB free / 3905 MB total
processes: 160
served by: pid 34558
$ 
```

圖 3：同時 10 個 client。 行程樹有 1 個父行程和 10 個子行程，port 9000 上有 10 條已建立的連線。

```
[pid 34694] done
[pid 34701] done
[pid 34698] done
[pid 34696] done
[pid 34702] done
[pid 34699] done
[pid 34700] done
[pid 34697] done
[pid 34695] done
[pid 34693] done


$ # 3. 同時 10 個 client：1 個父行程 + 10 個子行程、10 條連線
$ ./test10.sh
=== server 行程樹（1 個父行程 + 10 個子行程）===
    PID    PPID STAT CMD
  34516   34010 S+   ./server
  34693   34516 S+    \_ ./server
  34694   34516 S+    \_ ./server
  34695   34516 S+    \_ ./server
  34696   34516 S+    \_ ./server
  34697   34516 S+    \_ ./server
  34698   34516 S+    \_ ./server
  34699   34516 S+    \_ ./server
  34700   34516 S+    \_ ./server
  34701   34516 S+    \_ ./server
  34702   34516 S+    \_ ./server
=== port 9000 上已建立的連線 ===
Recv-Q        Send-Q              Local Address:Port               Peer Address:Port         Process
0             0                       127.0.0.1:9000                  127.0.0.1:56872
0             0                       127.0.0.1:9000                  127.0.0.1:56890
0             0                       127.0.0.1:9000                  127.0.0.1:56826
0             0                       127.0.0.1:9000                  127.0.0.1:56886
0             0                       127.0.0.1:9000                  127.0.0.1:56824
0             0                       127.0.0.1:9000                  127.0.0.1:56904
0             0                       127.0.0.1:9000                  127.0.0.1:56916
0             0                       127.0.0.1:9000                  127.0.0.1:56844
0             0                       127.0.0.1:9000                  127.0.0.1:56922
0             0                       127.0.0.1:9000                  127.0.0.1:56858
=== 10 個 client 全部完成 ===
$
```

圖 4：一般編譯，設定 `DEBUG_LOG=1` 也不印 debug 訊息。

```
$ DEBUG_LOG=1 ./server
server pid 35248 listening on port 9000 (robust=1)
[pid 35313] serving 127.0.0.1:50726
[pid 35313] done









$ # 7. 除錯 log：一般編譯，就算設 DEBUG_LOG=1 也不印
$ DEBUG_LOG=1 ./client 127.0.0.1
hostname : lima-nsp
kernel   : Linux 6.8.0-142-generic
uptime   : 933 seconds
load avg : 0.09 0.12 0.11
memory   : 2181 MB free / 3905 MB total
processes: 160
served by: pid 35313
$
```

圖 5：`make DEBUG=1` 編譯並設定 `DEBUG_LOG=1`，server 與 client 都印出 `[DEBUG pid ...]`。

```
$ DEBUG_LOG=1 ./server
server pid 35554 listening on port 9000 (robust=1)
[pid 35620] serving 127.0.0.1:53880
[DEBUG pid 35620] request: sysinfo
[DEBUG pid 35620] sent 180 bytes
[pid 35620] done
```

```
$ # 8. 除錯 log：make DEBUG=1 編譯 + DEBUG_LOG=1 執行，才印出 [DEBUG]
$ make clean; make DEBUG=1
rm -f libnsp.so server client
gcc -Wall -DDEBUG -fPIC -shared -o libnsp.so nsp.c
gcc -Wall -DDEBUG -o server server.c -L. -lnsp -Wl,-rpath,'$ORIGIN'
gcc -Wall -DDEBUG -o client client.c -L. -lnsp -Wl,-rpath,'$ORIGIN'
$ DEBUG_LOG=1 ./client 127.0.0.1
[DEBUG pid 35618] connected to 127.0.0.1:9000, waiting 0 seconds
hostname : lima-nsp
kernel   : Linux 6.8.0-142-generic
uptime   : 949 seconds
load avg : 0.07 0.11 0.11
memory   : 2174 MB free / 3905 MB total
processes: 160
served by: pid 35620
$
```

圖 6：封包擷取。 欄位依序為封包編號、來源埠、目的埠、TCP 旗標（S=SYN、A=ACK、P=PSH、F=FIN）、資料長度；逐封包說明見第五節。

```
$ ./server
server pid 35803 listening on port 9000 (robust=1)
[pid 36078] serving 127.0.0.1:55602
[pid 36078] done
```

```
$ # 9. 抓封包：tcpdump 錄 loopback，tshark 看交握、請求、回應、揮手
$ sudo tcpdump -i lo -nn -w /tmp/midterm.pcap tcp port 9000 2>/dev/null &
[1] 35952
$ ./client 127.0.0.1 > /dev/null; sleep 1; sudo pkill tcpdump
[1]+  Exit 1                  sudo tcpdump -i lo -nn -w /tmp/midterm.pcap tcp port 9000 2> /dev/null
$ tshark -r /tmp/midterm.pcap -T fields -e frame.number -e tcp.srcport -e tcp.dstport -e tcp.flags.str -e tcp.
len
1       47002   9000    ··········S·    0
2       9000    47002   ·······A··S·    0
3       47002   9000    ·······A····    0
4       47002   9000    ·······AP···    4
5       9000    47002   ·······A····    0
6       47002   9000    ·······AP···    7
7       9000    47002   ·······A····    0
8       9000    47002   ·······AP···    4
9       47002   9000    ·······A····    0
10      9000    47002   ·······AP···    180
11      47002   9000    ·······A····    0
12      9000    47002   ·······A···F    0
13      47002   9000    ·······A···F    0
14      9000    47002   ·······A····    0
$
```

圖 7：防護 1 對照。 關閉防護時，3 個 client 之後留下 3 個 `Z` 狀態的 `<defunct>`；開啟防護時，服務過 11 個 client 後只剩父行程。

```
$ ./server norobust
server pid 36573 listening on port 9000 (robust=0)
[pid 36767] serving 127.0.0.1:49060
[pid 36767] done
[pid 36769] serving 127.0.0.1:49074
[pid 36769] done
[pid 36771] serving 127.0.0.1:49080
[pid 36771] done
```

```
$ # 10. 關閉防護（./server norobust）：子行程沒人回收，變成殭屍 <defunct>
$ ./client 127.0.0.1 > /dev/null; ./client 127.0.0.1 > /dev/null; ./client 127.0.0.1 > /dev/null
$ ps -o pid,ppid,stat,cmd -C server
    PID    PPID STAT CMD
  36573   34010 S+   ./server norobust
  36767   36573 Z+   [server] <defunct>
  36769   36573 Z+   [server] <defunct>
  36771   36573 Z+   [server] <defunct>
$
```

```
[pid 34694] done
[pid 34701] done
[pid 34698] done
[pid 34696] done
[pid 34702] done
[pid 34699] done
[pid 34700] done
[pid 34697] done
[pid 34695] done
[pid 34693] done

$ # 4. 防護 1（回收子行程）：服務過 11 個 client，沒有殭屍行程
$ ps -o pid,ppid,stat,cmd -C server
    PID    PPID STAT CMD
  34516   34010 S+   ./server
$
```

圖 8：防護 2 對照。 關閉防護時，子行程 37005 已等了 7 秒仍在 `S` 狀態；開啟防護時，server 印出 `no valid request (timeout ...)` 並斷線，client 收到 `send failed`。

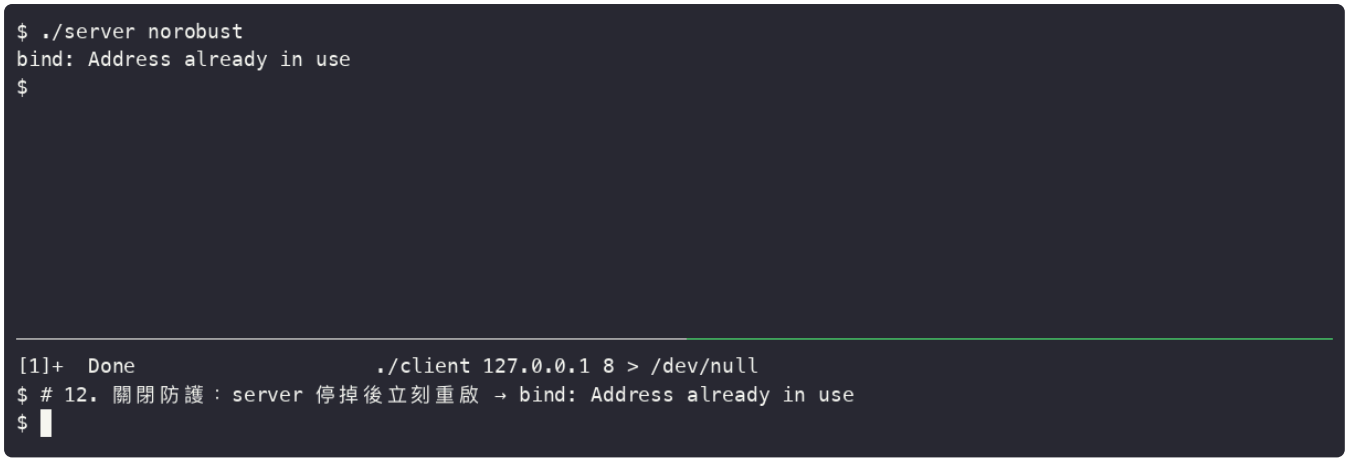
```
$ ./server norobust
server pid 36573 listening on port 9000 (robust=0)
[pid 36767] serving 127.0.0.1:49060
[pid 36767] done
[pid 36769] serving 127.0.0.1:49074
[pid 36769] done
[pid 36771] serving 127.0.0.1:49080
[pid 36771] done
[pid 37005] serving 127.0.0.1:38982
[pid 37005] done

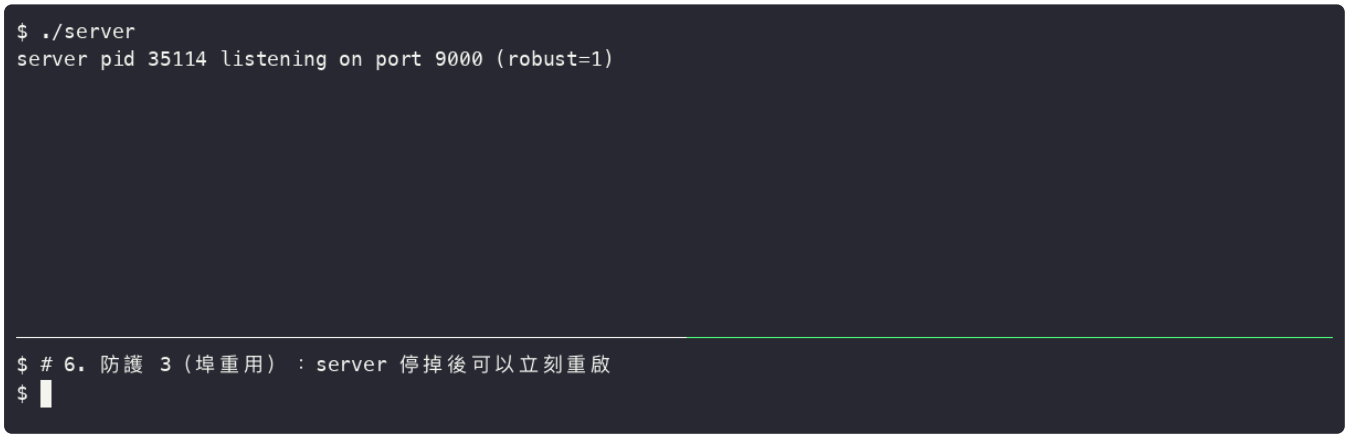
$ # 11. 關閉防護：client 8 秒不送請求，子行程一直卡著等（狀態 S）
$ ./client 127.0.0.1 8 > /dev/null &
[1] 37003
$ ps -o pid,ppid,stat,etime,cmd -C server
    PID    PPID STAT     ELAPSED CMD
  36573   34010 S+         00:22 ./server norobust
  36767   36573 Z+         00:19 [server] <defunct>
  36769   36573 Z+         00:19 [server] <defunct>
  36771   36573 Z+         00:19 [server] <defunct>
  37005   36573 S+         00:07 ./server norobust
$
```

```
[pid 34696] done
[pid 34702] done
[pid 34699] done
[pid 34700] done
[pid 34697] done
[pid 34695] done
[pid 34693] done
[pid 35012] serving 127.0.0.1:52968
[pid 35012] no valid request (timeout or disconnected)
[pid 35012] done

$ # 5. 防護 2（逾時）：client 連上後 8 秒不送請求，server 5 秒就斷線
$ ./client 127.0.0.1 8
send failed (server closed the connection)
$
```

圖 9：防護 3 對照。 關閉防護時，重啟 server 得到 `bind: Address already in use`；開啟防護時立刻重啟成功。





七、建置與執行方式

環境為 Ubuntu 24.04（arm64，Lima VM），gcc 編譯；在 `midterm` 資料夾執行 `make` 即可產生 `libnsp.so`、`server`、`client`。

指令	作用
`make`	一般編譯，debug log 不編進去
`make DEBUG=1`	把 debug log 編進去（切換前先 `make clean`）
`make clean`	刪除編譯產物
`./server`	啟動 server，開啟防護
`./server norobust`	啟動 server，關閉防護（對照用）
`./client <IP> [等待秒數]`	連到 server 並取得系統資訊；等待秒數用來製造併發與逾時情境
`./test10.sh`	同時 10 個 client 的併發測試
`DEBUG_LOG=1 ./server`	執行期開啟 debug log

Makefile 用 `-fPIC -shared` 編出 `libnsp.so`，兩支程式以 `-L. -lnsp` 連結，並加上 `-Wl,-rpath,'$ORIGIN'`，執行時在執行檔所在的資料夾尋找函式庫，不必設定 `LD_LIBRARY_PATH`。

八、完整原始碼（附錄）

共 6 個檔案，387 行（含註解）。

nsp.h

```
/* nsp.h：client 與 server 共用的常數與函式宣告，函式實作在 nsp.c（編成 libnsp.so）
*/

#define PORT 9000      /* server 監聽的埠號 */
#define MAX_MSG 4096   /* 一則訊息的最大長度（位元組） */

int send_msg(int fd, const char *data, int len);
int recv_msg(int fd, char *buf, int size);
void log_debug(const char *fmt, ...);
```

nsp.c

```
/* nsp.c：client 與 server 共用的工具函式，編譯成動態函式庫 libnsp.so */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <stdint.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include "nsp.h"

/*
 * write_all：把 buf 的 len 個位元組全部送出。
 *   send() 一次可能只送出一部分，所以用迴圈送到全部送完為止。
 *   MSG_NOSIGNAL：對方已斷線時回傳錯誤，而不是收到 SIGPIPE 讓整個行程被殺掉。
 * 輸入：fd 已連線的 socket；buf 要送的資料；len 要送的位元組數
 * 輸出：全部送完回傳 0；失敗回傳 -1
 */
static int write_all(int fd, const char *buf, int len)
{
    int done = 0;
```

```
    int n;

    while (done < len) {
        n = send(fd, buf + done, len - done, MSG_NOSIGNAL);
        if (n <= 0) {
            return -1;
        }
        done = done + n;
    }
    return 0;
}

/*
 * read_all：從 fd 讀滿 len 個位元組到 buf。
 *   recv() 一次可能只讀到一部分，所以用迴圈讀到讀滿為止。
 * 輸入：fd 已連線的 socket；buf 存放資料的緩衝區；len 要讀的位元組數
 * 輸出：讀滿回傳 0；對方斷線、逾時或出錯回傳 -1
 */
static int read_all(int fd, char *buf, int len)
{
    int done = 0;
    int n;

    while (done < len) {
        n = recv(fd, buf + done, len - done, 0);
        if (n <= 0) {
            return -1;
        }
        done = done + n;
    }
    return 0;
}

/*
 * send_msg：送出一則訊息。
 *   訊息格式：[4 位元組資料長度（網路位元組順序）][資料]
 * 輸入：fd 已連線的 socket；data 要送的資料；len 資料長度
 * 輸出：成功回傳 0；失敗回傳 -1
 */
int send_msg(int fd, const char *data, int len)
{
    uint32_t net_len = htonl(len);

    if (write_all(fd, (const char *)&net_len, 4) < 0) {
        return -1;
    }
```

```
    return write_all(fd, data, len);
}

/*
 * recv_msg：收一則 send_msg 送來的訊息，並在資料結尾補 '\0'，方便當字串使用。
 * 輸入：fd 已連線的 socket；buf 存放資料的緩衝區；size 緩衝區大小
 * 輸出：成功回傳資料長度；對方斷線、逾時、或資料長度超過緩衝區回傳 -1
 */
int recv_msg(int fd, char *buf, int size)
{
    uint32_t net_len;
    int len;

    if (read_all(fd, (char *)&net_len, 4) < 0) {
        return -1;
    }
    len = ntohl(net_len);
    if (len < 0 || len >= size) {
        return -1;
    }
    if (read_all(fd, buf, len) < 0) {
        return -1;
    }
    buf[len] = '\0';
    return len;
}

/*
 * log_debug：把除錯訊息印到 stderr，用法和 printf 相同。
 *   編譯期開關：用 make DEBUG=1 編譯時函式才有內容，否則是空函式，什麼都不印。
 *   執行期開關：執行時設定環境變數 DEBUG_LOG=1 才會真的印出。
 * 輸入：fmt 格式字串；... 對應 fmt 的參數
 * 輸出：無
 */
void log_debug(const char *fmt, ...)
{
#ifdef DEBUG
    va_list args;
    char *env = getenv("DEBUG_LOG");

    if (env == NULL || strcmp(env, "1") != 0) {
        return;
    }
    fprintf(stderr, "[DEBUG pid %d] ", getpid());
    va_start(args, fmt);
    vfprintf(stderr, fmt, args);
```

```
    va_end(args);
    fprintf(stderr, "\n");
#endif
}
```

server.c

```
/* server.c：每接到一個 client 連線就 fork 一個子行程服務它，回傳本機系統資訊 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/socket.h>
#include <sys/sysinfo.h>
#include <sys/utsname.h>
#include <sys/time.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include "nsp.h"

#define TIMEOUT_SEC 5   /* 防護 2：等待 client 請求的最長秒數 */

/*
 * reap_children：SIGCHLD 的處理函式，子行程結束時由核心呼叫。
 *   用 waitpid 收掉所有已結束的子行程，避免它們變成殭屍行程。
 * 輸入：sig 訊號編號（不使用）
 * 輸出：無
 */
static void reap_children(int sig)
{
    while (waitpid(-1, NULL, WNOHANG) > 0) {
    }
}

/*
 * build_sysinfo：查詢本機系統資訊，寫成一段文字。
 * 輸入：buf 存放文字的緩衝區；size 緩衝區大小
 * 輸出：回傳寫入的文字長度（不含結尾 '\0'）
 */
static int build_sysinfo(char *buf, int size)
{
    struct utsname u;
    struct sysinfo s;
```

```
    uname(&u);
    sysinfo(&s);
    return snprintf(buf, size,
                    "hostname : %s\n"
                    "kernel   : %s %s\n"
                    "uptime   : %ld seconds\n"
                    "load avg : %.2f %.2f %.2f\n"
                    "memory   : %lu MB free / %lu MB total\n"
                    "processes: %d\n"
                    "served by: pid %d\n",
                    u.nodename,
                    u.sysname, u.release,
                    s.uptime,
                    s.loads[0] / 65536.0, s.loads[1] / 65536.0, s.loads[2] /
65536.0,
                    s.freeram * s.mem_unit / 1024 / 1024, s.totalram *
s.mem_unit / 1024 / 1024,
                    s.procs,
                    getpid());
}

/*
 * handle_client：子行程服務一個 client：收請求、回應、結束。
 * 輸入：fd 與 client 連線的 socket；robust 1 表示開啟防護，0 表示關閉
 * 輸出：無
 */
static void handle_client(int fd, int robust)
{
    char req[MAX_MSG];
    char reply[MAX_MSG];
    int len;
    struct timeval tv;

    if (robust == 1) {
        /* 防護 2：client 超過 TIMEOUT_SEC 秒沒送請求，recv 就回傳錯誤，子行程不會永
遠卡住 */
        tv.tv_sec = TIMEOUT_SEC;
        tv.tv_usec = 0;
        setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
    }
    if (recv_msg(fd, req, MAX_MSG) < 0) {
        printf("[pid %d] no valid request (timeout or disconnected)\n",
getpid());
        return;
    }
```

```
    log_debug("request: %s", req);
    if (strcmp(req, "sysinfo") == 0) {
        len = build_sysinfo(reply, MAX_MSG);
    } else {
        len = snprintf(reply, MAX_MSG, "unknown request: %s\n", req);
    }
    if (send_msg(fd, reply, len) < 0) {
        printf("[pid %d] send failed (client disconnected)\n", getpid());
        return;
    }
    log_debug("sent %d bytes", len);
}

/*
 * main：流程──建立監聽 socket，每接到一個連線就 fork 一個子行程去服務。
 *   用法：./server             開啟防護機制
 *         ./server norobust    關閉防護機制（示範用）
 */
int main(int argc, char *argv[])
{
    int robust = 1;
    int listen_fd;
    int client_fd;
    int opt = 1;
    pid_t pid;
    struct sockaddr_in addr;
    struct sockaddr_in peer;
    socklen_t peer_len;
    struct sigaction sa;

    if (argc > 1 && strcmp(argv[1], "norobust") == 0) {
        robust = 0;
    }
    /* stdout 改成每行立刻輸出，避免 fork 時子行程複製到還沒印出的內容而重複印 */
    setvbuf(stdout, NULL, _IOLBF, 0);

    if (robust == 1) {
        /* 防護 1：子行程結束時自動回收，避免殭屍行程；SA_RESTART 讓被打斷的 accept 自
動重來 */
        memset(&sa, 0, sizeof(sa));
        sa.sa_handler = reap_children;
        sa.sa_flags = SA_RESTART;
        sigaction(SIGCHLD, &sa, NULL);
    }

    listen_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (robust == 1) {
        /* 防護 3：允許重用還在 TIME_WAIT 狀態的埠，server 重啟時 bind 不會失敗 */
        setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    }
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);
    if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        return 1;
    }
    listen(listen_fd, 16);
    printf("server pid %d listening on port %d (robust=%d)\n", getpid(),
PORT, robust);

    while (1) {
        peer_len = sizeof(peer);
        client_fd = accept(listen_fd, (struct sockaddr *)&peer, &peer_len);
        if (client_fd < 0) {
            perror("accept");
            continue;
        }
        pid = fork();
        if (pid == 0) {
            close(listen_fd);
            printf("[pid %d] serving %s:%d\n", getpid(),
inet_ntoa(peer.sin_addr), ntohs(peer.sin_port));
            handle_client(client_fd, robust);
            close(client_fd);
            printf("[pid %d] done\n", getpid());
            exit(0);
        }
        close(client_fd);
    }
}
```

client.c

```
/* client.c：連到 server，請求系統資訊並印出 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
```

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include "nsp.h"

/*
 * main：流程──連到 server，等待指定秒數，送出 "sysinfo" 請求，印出回應。
 *   用法：./client <server IP> [等待秒數]
 *   等待秒數：連上後先等幾秒才送請求，用來製造「多個連線同時存在」與「逾時」的情境。
 */
int main(int argc, char *argv[])
{
    int fd;
    int wait_sec = 0;
    struct sockaddr_in addr;
    char reply[MAX_MSG];

    if (argc < 2) {
        printf("usage: %s <server_ip> [wait_seconds]\n", argv[0]);
        return 1;
    }
    if (argc > 2) {
        wait_sec = atoi(argv[2]);
    }

    fd = socket(AF_INET, SOCK_STREAM, 0);
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, argv[1], &addr.sin_addr) != 1) {
        printf("invalid IP address: %s\n", argv[1]);
        return 1;
    }
    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("connect");
        return 1;
    }
    log_debug("connected to %s:%d, waiting %d seconds", argv[1], PORT,
wait_sec);
    sleep(wait_sec);

    if (send_msg(fd, "sysinfo", 7) < 0) {
        printf("send failed (server closed the connection)\n");
        return 1;
    }
    if (recv_msg(fd, reply, MAX_MSG) < 0) {
        printf("no reply (server closed the connection)\n");
```

```
        return 1;
    }
    printf("%s", reply);
    close(fd);
    return 0;
}
```

Makefile

```
# 用法：make             一般編譯（除錯訊息不編進去）
#       make DEBUG=1     把除錯訊息編進去（切換前先 make clean）
#       make clean       刪除編譯產物

CFLAGS = -Wall

ifdef DEBUG
CFLAGS = -Wall -DDEBUG
endif

all: libnsp.so server client

# 共用函式庫：-fPIC 產生可重定位的程式碼，-shared 輸出成動態函式庫
libnsp.so: nsp.c nsp.h
        gcc $(CFLAGS) -fPIC -shared -o libnsp.so nsp.c

# -L. -lnsp 連結 libnsp.so；-rpath '$$ORIGIN' 讓執行時到執行檔所在的資料夾找
libnsp.so
server: server.c nsp.h libnsp.so
        gcc $(CFLAGS) -o server server.c -L. -lnsp -Wl,-rpath,'$$ORIGIN'

client: client.c nsp.h libnsp.so
        gcc $(CFLAGS) -o client client.c -L. -lnsp -Wl,-rpath,'$$ORIGIN'

clean:
        rm -f libnsp.so server client
```

test10.sh

```
#!/bin/bash
# 併發測試：同時啟動 10 個 client，每個連上後等 5 秒才送請求。
# 在這 5 秒內列出 server 的行程樹與連線，證明 10 個 client 同時被 10 個子行程服務。
# 用法：先在另一個視窗執行 ./server，再執行 ./test10.sh

for i in 1 2 3 4 5 6 7 8 9 10; do
    ./client 127.0.0.1 5 > /dev/null &
done
sleep 1

echo "=== server 行程樹（1 個父行程 + 10 個子行程）==="
ps -o pid,ppid,stat,cmd --forest -C server

echo "=== port 9000 上已建立的連線 ==="
ss -tn state established '( sport = :9000 )'

wait
echo "=== 10 個 client 全部完成 ==="
```